

Detection and Correction Process Modeling Considering the Time Dependency

Y.P. Wu, Q.P. Hu, M. Xie and S.H. Ng
Department of Industrial and Systems Engineering
National University of Singapore
(g0402773, g0305835, mxie, isensh)@nus.edu.sg

Abstract

Most of the models for software reliability analysis are based on reliability growth models which deal with the fault detection process only. In this paper, some useful approaches to the modeling of both software fault detection and fault correction processes are discussed. Since the estimation of model parameters in software testing is essential to give accurate prediction and help make the right decision about software release, the problem of estimating the parameters is addressed here. Taking into account the dependency between the fault correction process and the fault detection process, a new explicit formula for the likelihood function is derived and the maximum likelihood estimates are obtained under various time delay assumptions. An actual set of data from a software development project is used as an illustrative example. A Monte Carlo simulation is carried out to compare the predictive capability between the LSE method and the MLE method.

1. Introduction

During the last few decades, generalizations and extensions of software reliability growth models (SRGMs) continue to attract researchers in the field. Since software is widely used in our daily lives, the correct performance of software system becomes an important issue. Traditional SRGMs only consider the fault detection process in software testing, assuming perfect and immediate fault correction. However, this assumption may not be reasonable. For each fault detected, it is reported, diagnosed, verified, and then corrected. The time between detection and correction should not be neglected in practical software testing process. Recently, more and more researchers emphasized the great importance of the fault correction modeling ([1], [4], [12], and [19]). As a result, combined fault detection and correction modeling could present more practical models for

software testing process. In this paper, a systematic study on the fault detection and correction process is carried out, with the emphasis on the fault correction process. Various fault correction models are proposed considering different forms of the time delay.

Fitting a proposed model to the actual data of faults detection and correction involves estimating the model's parameters from the real test data sets. Once able to estimate those parameters, we can give accurate predictions to the future behavior of the fault detection and correction process, which will help software managers to allocate testing resources and study the software release problems. Maximum likelihood estimation (MLE) is one of the most popular estimation techniques. Parameters are estimated by solving a set of simultaneous equations. Up to now, only the Least Square Estimation (LSE) method has been applied in estimating the combined fault detection and correction process modeling [18], no Maximum Likelihood Estimation (MLE) method has been applied in the existing studies to estimate parameters in the fault detection and correction process; although it is well accepted that the MLE method is quite straightforward having many desired properties such as asymptotic normality, admissibility, robustness and consistency, and widely used to estimate the parameters for SRGMs ([5], [7], [10] and [16]). Taking into account the dependency between the FDP and the FCP, we consider the issue of applying the MLE method to the combined fault detection process (FDP) and fault correction process (FCP) modeling from theoretical and experimental perspectives.

In the next section, some useful approaches to the modeling of both software fault detection and fault correction processes are discussed. Considering the dependency between the FDP and the FCP, a new explicit formula for the likelihood function of the combined fault detection and correction process is derived, with the ML estimates obtained under various time delay assumptions. A real dataset is applied and a comparison of prediction performance between the MLE method and the LSE method is illustrated. No

single comparison is adequate to determine the method with better prediction performance; therefore, a Monte Carlo simulation analysis is carried out.

2. Literature review

Software reliability, defined as the probability of failure-free software operation for a specified period of time in a specified environment [6], is supposed to be a good measurement to quantify software failures. Lots of software reliability growth models (SRGMs) have been proposed to measure the software failure process successfully ([3], [13], and [15]), among them some are based on non-homogeneous Poisson process (NHPP) ([6], [8], [10] and [12]).

Traditional SRGMs only consider the fault detection process assuming perfect and immediate fault correction. Software fault-detection process $N(t)$ is usually assumed to follow a NHPP, in which the intensity function $\lambda_d(t)$ is time-dependent. Given $\lambda_d(t)$, the mean value function (MVF) $m_d(t)$ satisfies

$$m_d(t) = \int_0^t \lambda_d(s) ds \quad (1)$$

2.1. The Goel-Okumoto model

The GO-model [2] is one of the most influential NHPP software reliability models. The mean value function is given as

$$m_d(t) = a \cdot (1 - e^{-bt}), \quad a, b > 0 \quad (2)$$

where a is the number of faults that can be detected by the testing process, and b can be interpreted as the failure occurrence rate per fault.

2.2. Yamada delayed S-shaped model

The Yamada delayed S-shaped (DSS) model is an S-shaped curve for the cumulative number of detected faults.

$$m_d(t) = a \cdot [1 - (1 + bt)e^{-bt}], \quad a, b > 0 \quad (3)$$

with parameter a denoting the number of faults to be detected, and b corresponding to a fault detection rate.

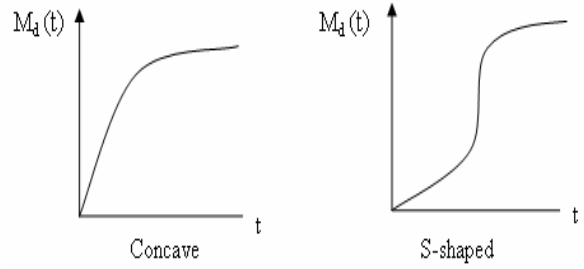


Figure 1. Two classes of MVF $m_d(t)$

To highlight the idea and approach in our study, we propose our fault detection and correction model based on G-O model as an example. Mainly there are two classes of $m_d(t)$ used to describe different fault detection processes: concave and S-shaped models, as illustrated in Figure 1. Concave $m_d(t)$ describes the fault detection process with exponential decreasing intensity. Differently, S-shaped $m_d(t)$ describes fault detection process with increasing-then-decreasing intensity, which can be interpreted as a learning process.

3. Modeling of the fault detection and correction process

Traditional SRGMs consider the fault detection process only, usually assuming the fault detected is corrected immediately. Schneidweind [9] first proposed the idea of modeling the fault correction process as a separate process following the fault detection process with a constant time delay. It was later highlighted in [17] where a time-dependent delay function was proposed. In [11], the time delay was assumed to be an exponentially distributed random variable. Xie et al [18] carried out a systematic study on fault detection and correction process, with the emphasis on fault correction process described by delayed detection process with random or deterministic delay.

In this paper, we give the parameter estimation and comparison based on the fault detection and correction models proposed by [18]. Similar to FDP models characterized with mean value function (MVF) $m_d(t)$, FCP models are characterized with MVF $m_c(t)$. The MVF of FCP models can be derived from $m_d(t)$ and the time delay as $m_c(t) = m_d(t - \Delta)$. To emphasize the fault correction modeling, G-O model characterized with MVF $m_d(t) = a \cdot (1 - e^{-bt})$, $a, b > 0$ is applied to the fault detection process for illustrative purpose.

3.1. Exponentially distributed time delay

The correction time approximately follows exponential distribution in many practical software testing projects [7]. Assuming the correction time for each detected fault is exponentially distributed with $\Delta \sim \exp(\mu)$, the fault correction MVF for GO-model is given as

$$m_c(t) = \begin{cases} a \cdot [1 - (1 + bt)e^{-bt}] & \mu = b \\ a \cdot \left[1 - \frac{\mu}{\mu - b} e^{-bt} + \frac{b}{\mu - b} e^{-\mu t} \right] & \mu \neq b \end{cases} \quad (4)$$

where $m_c(t)$ has the same form as the $m_d(t)$ for S-shaped NHPP model while $\mu = b$.

3.2. Normally distributed time delay

Assuming faults are of equal size, then we can model the time delay as a normally distributed variable with mean μ and variance σ^2 , the fault correction MVF for GO-model is given as

$$m_c(t) = -ae^{-bt + \mu b + b^2 \sigma^2 / 2} (\Phi(t, b\sigma^2 + \mu, \sigma) - \Phi(0, b\sigma^2 + \mu, \sigma)) + a(\Phi(t, \mu, \sigma) - \Phi(0, \mu, \sigma)) \quad (5)$$

3.3. Gamma distributed time delay

To provide more flexible modeling of the correction processes, some extended distributions can be used for the correction time. One possible distribution is the Gamma distribution, which is the generalized form of the exponential distribution. Assume $\Delta \sim \Gamma(\alpha, \beta)$, the fault correction MVF for GO-model is given as

$$m_c(t) = a\Gamma(t, \alpha, \beta) - \frac{ae^{-bt}}{(1 - b\beta)^\alpha} \Gamma(t, \alpha, \frac{\beta}{1 - b\beta}) \quad (6)$$

4. Maximum likelihood estimation for the fault detection and correction process

Fitting a proposed model to the actual data of faults detection and correction involves estimating the model's parameters from the real test data sets. Once able to estimate those parameters, we can give accurate predictions to the future behavior of the fault detection and correction process, which will help software managers to allocate testing resources and study the software release problems. Below we consider using MLE method to estimate parameters for the fault detection and correction process modeling.

With respect to the fault detection and correction process, the fault correction process can be modeled by $m_c(t_i) = m_d(t_i - \Delta_i)$, with the time delay $\Delta_i, i = 1, 2, \dots \sim \Delta$ satisfying a certain probability distribution. Denoting n_i and m_i as the cumulative number of faults detected and corrected by time t_i respectively, and $n_0 = m_0 = 0$. Assuming the fault detection process is NHPP, we can see n_i depends on m_i .

Denote $P(n_i, m_i)$ as the probability of detecting n_i faults and correcting m_i faults by time t_i . $P_d(\cdot)$ as the probability distribution function for detection process, and $P_c(\cdot)$ as the probability distribution function for correction process. In this dependent case, conditioning on the corrected number of faults, we can rewrite the probability of detecting n_i faults and correcting m_i faults by time t_i at the very first step, then, denoting $\theta_0 \in \Theta \subset R^m$ as the parameters in the fault detection and correction modeling, then the joint density of the detected and corrected fault counts over the given partition can be obtained and finally, the likelihood function can be obtained with θ_0 replaced by θ :

$$\begin{aligned} L &= f(n_i, m_i, i = 1, \dots, k | \theta) \\ &= \prod_{\substack{i=1, \dots, k_i \\ m_i > n_{i-1}}} e^{-[m_d(t_i) - m_d(t_{i-1})] \theta} \frac{[m_d(t_i) - m_c(t_i)] \theta^{n_i - m_i}}{(n_i - m_i)!} \frac{[m_c(t_i) - m_d(t_{i-1})] \theta^{m_i - n_{i-1}}}{(m_i - n_{i-1})!} \\ &\times \prod_{\substack{i=1, \dots, k_i \\ m_i < n_{i-1}}} e^{-[m_d(t_i) - m_d(t_{i-1})] \theta} \frac{[m_d(t_i) - m_d(t_{i-1})] \theta^{n_i - n_{i-1}}}{(n_i - n_{i-1})!} \\ &\cdot e^{-[m_c(t_i) - m_c(t_{i-1})] \theta} \frac{[m_c(t_i) - m_c(t_{i-1})] \theta^{m_i - m_{i-1}}}{(m_i - m_{i-1})!} \end{aligned} \quad (7)$$

A general form of the likelihood function for the combined fault detection and correction process is given in equation (7). Under various time delay assumption, the maximum likelihood (ML) estimates of θ_0 can be obtained as $\hat{\theta} = \arg \max_{\theta \in \Theta} \hat{L}(\theta)$. Once the estimates of all the parameters are obtained, we can use the invariance property of the MLEs to estimate other reliability measures by replacing the respective parameters according to their corresponding ML estimates.

5. A case study

5.1. Parameter estimation

To illustrate the application, GO-model paired with various fault correction models (due to various time

delay distribution) are applied based on an actual data set from [18]. The proposed MLE approach is carried out to estimate parameters taking into account the dependency between the fault detection data and the fault correction data.

5.1.1. Exponentially distributed time delay. Assuming the correction time for each detected fault is exponentially distributed with $\Delta \sim \exp(\mu)$, the ML estimates for a, b and μ are: $\hat{a}=165$, $\hat{b}=0.12$, $\hat{\mu}=1.63$.

5.1.2. Normally distributed time delay. Assuming faults are of equal size, the time delay can be modeled as a normally distributed variable with mean μ and variance σ^2 , the ML estimates of a, b, μ and σ are: $\hat{a}=176$, $\hat{b}=0.1$, $\hat{\mu}=0.53$, $\hat{\sigma}=0.1$.

5.1.3. Gamma distributed time delay. Assume the random time delay to be Gamma distributed $\Delta \sim \Gamma(\alpha, \beta)$, the ML estimates of a, b, α and β are: $\hat{a}=166$, $\hat{b}=0.12$, $\hat{\alpha}=0.73$, $\hat{\beta}=0.83$.

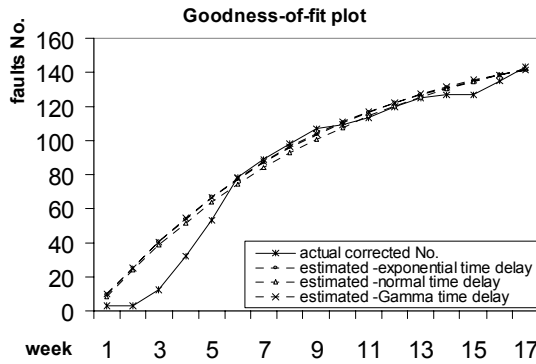


Figure 2. Plot of the goodness-of-fit for the FCP under various time delay forms

From Figure 2, we can see from 5th point to the 17th point, the ML estimates can fit the actual data well. Actually, software managers do not emphasize having a good fitness to the data; instead, they would prefer to have a parameter method with good prediction performance. From this perspective, we further consider the capability of the ML estimates to predict the future behavior.

5.2. Prediction performance

For a software manager, it is essential to be able to predict the future behavior of the fault detection and

correction process. It means a lot to a software manager to have a good fitness to the data at the later stage of the software testing, because the further the software testing goes, the more expensive it will be. In addition, the prediction is important for the allocation of further testing resources and for the study of software release problems.

Therefore, we further consider the predictive capability of the ML estimates here. Assuming exponential time delay, we show predictions five steps ahead, i.e., using only the data of the first 12 weeks to predict the next five weeks. The predicted value is then compared with the real observed dataset. We define $RE = \frac{m(t_i) - N_i}{N_i}$, where N_i is the faults observed by time t_i . RE is called predictive validity. The comparison criterion is the mean of relative errors

(MRE), defined as $MRE = \frac{1}{l} \sum_{i=1}^l \left| \frac{m(t_i) - N_i}{N_i} \right|$, which can show the capability of the model to predict the fault behavior ([7]). Moreover, a lower value of MRE means a better prediction performance.

Based on an actual data set from [18], with the dataset of the first 12 weeks provided, the ML estimates of those parameters are obtained as $\hat{a}=163$, $\hat{b}=0.12$, $\hat{\mu}=1.39$. Predictions of the rest five weeks are compared with the actual observations as plotted in Figure 3. The predictions of the rest five weeks using the MLE method show that: for the detection process, MRE is 0.026; for the correction process, MRE is 0.027. The average MRE is 0.026.

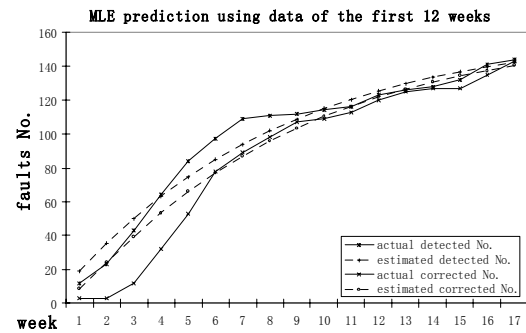


Figure 3. MLE prediction using data of the first 12 weeks

To have a general idea of the prediction performance, we compared it with the one using the LS estimates. With the dataset of the first 12 weeks provided, the LS estimates of those parameters are obtained as $\hat{a}=156$, $\hat{b}=0.14$, $\hat{\mu}=0.58$. Predictions of

the rest five weeks are compared with the actual observations as plotted in Figure 4. The prediction of the rest five weeks using the LSE method show that: for the detection process, MRE is 0.053; for the correction process, MRE is 0.025. The average MRE is 0.039.

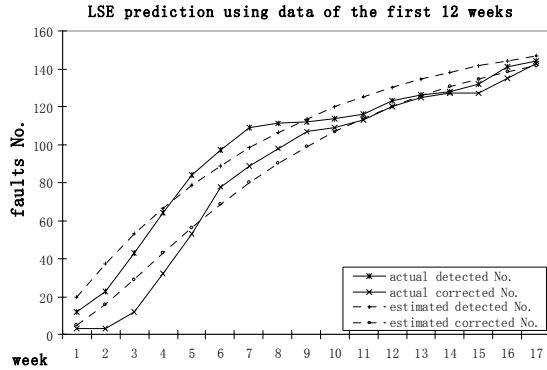


Figure 4. LSE prediction using data of the first 12 weeks

In this numerical example, after comparing Figure 3 and Figure 4, we can see the prediction performance of the MLE method is better than the LSE method based on this numerical example. The LSE method actually has more severe deviations to the FDP compared with the prediction results of the MLE method.

It is reasonable that the MRE criterion shows that the MLE method has good prediction capability the combined FDP and FCP, as the MLE method takes into account the dependency between the FDP and the FCP while the LSE method does not.

6. Monte Carlo simulation study

Reliability modeling ultimately requires good data, but software projects do not always collect data sets that are comprehensive for model application, therefore, there is need for dynamic simulation to analyze software reliability models. Consider from this perspective, we aim at studying the predictive capability of the ML estimates in a more general way. Instead of using only one dataset for illustration, we simulate a large number of datasets using Monte Carlo simulation method.

6.1. Simulation method

Several simulation code generation approaches for software reliability dataset have been proposed by Tausworthe [14]. Based on that idea of generating

code program, we considered writing a code of programming to simulate data sets of fault detection and correction in our modeling. Since the real data for detection and correction process is limited, in order to provide the reader a firm understanding of the prediction performance of MLE method, we carried out an empirical study by Monte Carlo simulation based on G-O model using MATLAB 6.5 version software. The steps are as below:

(1) Generate two sequences of uniformly distributed data $\{u_i, i=1, \dots, N\}$ $\{v_i, i=1, \dots, N\}$; we choose $N=300$.

(2) Generate the inter-arrival time of HPP event as $t_i = -\log u_i, i=1, \dots, N$;

(3) From step (1) and (2) we get $S_0 = 0, S_i = S_{i-1} + t_i, i=1, \dots, N$ as the arrival time of HPP event;

(4) $s_i = m_d^{-1}(S_i), i=1, \dots, N$ is the simulation of NHPP data for fault detection process, where $m_d(t)$ is the MVF of G-O model.

(5) Considering $m_c(t) = m_d(t - \Delta)$ we can generate the arrival time of the NHPP data of correction process as $s'_i = s_i + t'_i, i=1, \dots, N$, where the exponentially distributed time delay $\Delta \sim \exp(\mu)$ is simulated as $t'_i = -\log v_i / \mu, i=1, \dots, N$, here μ is the parameter for the exponential distribution;

(6) For $j=1, \dots, 17, i=1, \dots, N$, if $s_i < j, d_i = d_i + 1$, if $s'_i < j, c_i = c_i + 1$.

(7) $d_j, j=1, \dots, 17$ and $c_j, j=1, \dots, 17$ are the cumulative detected faults No. and corrected faults No. separately.

6.2. A case study

We choose $\hat{a}=165, \hat{b}=0.12, \hat{\mu}=1.6, N=300$ and simulate several sets of fault detection and correction data. To have a firm understanding of the predictive capability of the ML estimates, we make a comparison between the MLE predictions and the LSE predictions. Using the same method mentioned in section 5.2, based on the simulated datasets of the first 12 weeks, we calculate the ML estimates and the LS estimates separately. Using those estimated parameters, we give predictions to the next five weeks. The criterion to judge the prediction performance is the MRE defined the same as above.

6.2.1. Simulation of 10 data sets. At the beginning, we simulate 10 sets of data to have a general idea of the prediction performance of those two methods; Figure 5 shows the results of the Monte Carlo study

by plotting the curve of the average value of RE of the predicted values for the rest five weeks, using the MLE method and the LSE method with 10 sets of simulated data.

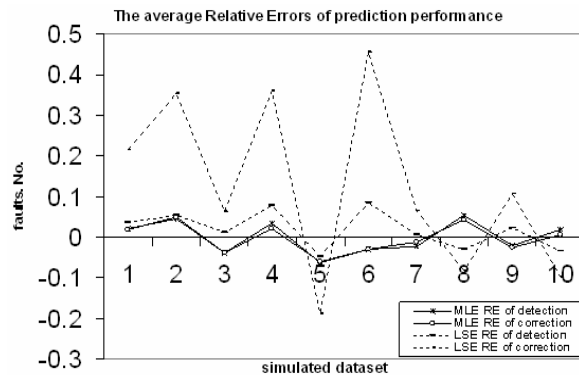


Figure 5. Plot of the average of RE

6.2.2. Simulation of 120 data sets. 120 datasets of the fault detection and correction process are simulated with the initial value $\mu = 1.6$, $\mu = 2$, $\mu = 3$ separately. MRE is used as a criterion to judge the prediction performance. If the MRE of both detected fault data and corrected fault data of method 1 are less than that of method 2, we consider method 1 give better predictions, otherwise, method 2 is considered to give better predictions. If any other cases, we consider the two methods incomparable. MRED is used to denote the MRE of detected fault data, and MREC is used to denote the MRE of corrected fault data.

Table 1. The MRE of predicted value simulating 120 datasets (the MLE method)

MLE method		MRED	MREC	Percentile
$\mu = 1.6$	mean	0.038	0.034	52.70 %
	std.	0.023	0.021	
$\mu = 2$	mean	0.026	0.023	60.20 %
	std.	0.022	0.019	
$\mu = 3$	mean	0.027	0.024	72 %
	std.	0.02	0.02	

Table 2. The MRE of predicted value simulating 120 datasets (the LSE method)

LSE method		MRED	MREC	Percentile
$\mu = 1.6$	mean	0.041	0.036	47.30 %
	std.	0.03	0.026	
$\mu = 2$	mean	0.028	0.025	39.80 %
	std.	0.025	0.022	
$\mu = 3$	mean	0.037	0.034	28 %
	std.	0.027	0.025	

Table 1 and Table 2 compare the mean value and the standard deviation of MRED and MREC between the MLE method and the LSE method. The Monte Carlo simulation results show that the MLE method has a lower value of mean of MRE for both detection process and correction process in the prediction performance, and the standard deviation of MRE for both detection process and correction process is less than that of the LSE method, which means a more stable prediction capability. More than half of the simulation results show that the MLE give better predictions compared with the LSE method. From the perspective of a software manager, more accurate prediction can help monitoring the software process much better. The results show that the ML estimates have a fairly accurate prediction capability compared with the LS estimates.

7. Conclusion

In this paper, a systematic study on the fault detection and correction process is carried out. Based on the modeling of the combined software fault detection and correction process, a new explicit formula for the likelihood function is derived, taking into account the dependency between detected faults and corrected faults, with the ML estimates obtained under various time delay assumptions. A real dataset is applied and the prediction performance of the MLE method is illustrated. Since no single comparison is adequate to determine which method can give better prediction performance; therefore, a Monte Carlo simulation analysis is carried out. Experimental results of the simulation analysis show that the ML estimates have a fairly accurate prediction capability compared with the LS estimates. The approach in our study can be further extended to general SRGMs considering the fault detection and correction process.

8. References

- [1] Bustamante, A.S. and Bustamante, B.S., "Multinomial-exponential reliability function: a software reliability model", *Reliability Engineering and System Safety*, vol. 79, 2003, pp. 281-288.
- [2] Goel, A.L. and Okumoto K., "Time-dependent error-detection rate model for software reliability and other performance measures", *IEEE transactions on reliability*, vol. 28, 1979, pp. 206-211.
- [3] Huang, C.Y., Lyu, M.R. and Kuo, S.Y., "A unified scheme of some NHPP models for software reliability estimation", *IEEE Transactions on Software Engineering*, vol. 29, no. 3, 2003, pp. 261-269.

- [4] Huang, C.Y. and Lin C. D., "Software reliability analysis by considering fault dependency and debugging time lag", *IEEE Transactions on Reliability*, in press
- [5] Inoue S. and Yamada S., "Testing-coverage dependent software reliability growth modeling", *International journal of reliability, quality and safety engineering*, vol. 11, no. 4, 2004, pp. 303-312.
- [6] Lyu, M.R., *Handbook of Software Reliability Engineering*. McGraw-Hill, 1996.
- [7] Musa, J.D., Iannino, A. and Okumoto, K., *Software reliability: measurement, prediction, application*. McGraw-Hill, New York., 1987
- [8] Pham, H., *Software Reliability*. Springer-Verlag, 2000
- [9] Schneidewind N.F., "Analysis of error processes in computer software", *Proceedings of International Conference on Reliable Software, IEEE Computer Society*, 1975, pp. 337-346.
- [10] Schneidewind N.F., "Software reliability model with optimal selection of failure data", *IEEE transactions on software engineering*, vol. 19, no. 11, 1993, pp. 1095-1104
- [11] Schneidewind N.F., "Modeling the fault correction process", *Proceedings of the twelfth international symposium on software reliability engineering*, 2001, pp. 185-190.
- [12] Schneidewind N.F., "Fault correction profiles", *Proceedings of the 14th international symposium on software reliability engineering*, 2003, pp. 257-267.
- [13] Tamura, Y. and Yamada, S., "A flexible stochastic differential equation model in distributed development environment", *European Journal of Operational Research*. vol. 168, no. 1, 2006, pp. 143-152.
- [14] Tausworthe, R. C. and Lyu, M.R., "A Generalized Technique for Simulating Software Reliability", *IEEE Software*, vol. 13, no. 2, 1996, pp. 77-88.
- [15] Teng X. L. and Pham H., "A software-reliability growth model for N-version programming systems", *IEEE transactions on reliability*, vol. 51, no. 3. 2002, pp. 311-321.
- [16] Xie, M., *Software Reliability Modeling*. World Scientific Publishing Company, 1991
- [17] Xie, M. and Zhao, M., "The Schneidewind software reliability model revisited", *Proceedings of the 3rd International Symposium on Software Reliability Engineering*, 1992, pp. 184-192.
- [18] Xie, M., Hu, Q.P., Wu Y.P. and Ng S.H., "A Study of Software Fault Detection and Correction Modeling", *Quality and reliability engineering international*, in press
- [19] Zhang, X.M., Teng, X.L. & Pham, H., "Considering fault removal efficiency in software reliability assessment", *EE Transactions on Systems, Man and Cybernetics: Part A – Systems and Humans*, vol. 33, 2003, pp.114-120.